Loops and trace tables

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

List the visits. Trace before and after. Place the output.

Inspect an accumulator loop without including its excluded stop or resetting past state.

Keep one habit: Three-value warm-up → four-value graphic → six state/output cases → selected range → portfolio trace.

MAIN LESSON · SECTION 1

Which value is visited, and what is the total after it?

> Track the loop input, accumulator and output events separately.

Your learning goals

I can trace variable values before and after each **loop iteration**.

I can use the excluded stop value in range correctly.

I can explain **initialization** and test zero one and several **iteration**s.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Trace variable values before and after each **loop iteration**
- Use the excluded stop value in range correctly
- Explain **initialization** and test zero one and several **iteration**s
- Starting with total = 2, what does total = total + 3 store?

> **My starting explanation**

Your end goal

Trace **loop**s and **trace table**s and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Repeat an addition over a declared range

> The start is included and stop excluded for this positive-step integer range.

Read the running-total task

Imagine adding numbered counters: first 1, then 2, then 3. A **loop** can repeat the same addition with a different number each time. This week you will predict each change before running a short Python script.

Tracing repeated calculations prepares you to inspect the data **process**ing used in larger AI applications.

Identify loop body, colon and indentation

In for turn in range(1, 4), Python supplies 1, 2 and 3 in order. The start is included and the stop is excluded. The indented body runs once for each supplied value. The colon and four-space indentation mark that body.

This lesson uses integer range values with the default positive step of 1.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Initialise once, then update the existing total

> The table records what the code does; it is not another instruction.

Complete 1-through-3 accumulator script

```
total = 0
for turn in range(1, 4):
    total = total + turn
print(total)
```

Trace current value and total

State after each addition

TURN	TOTAL BEFORE	TOTAL AFTER
1	0	1
2	1	3
3	3	6

Initialize total to 0 before the loop. The unindented print runs once afterward.

Explain the final output and loop exit

The **output** is 6. Before the first **iteration**, total is 0. Each assignment reads the previous total and adds the current turn. After turn 3, the range is exhausted, so execution continues after the **loop**.

The table records state; it does not create extra instructions.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare an empty range with a repeated reset

> Zero, one and several iterations test different failure modes.

Predict the supported change

- Copy the worked script. Change the range to range(2, 5). Record turn, total before and total after each **iteration**. Predict the single **output**, then run and compare.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured variant

- Change: Replace range(1,5) with range(1,1).

▼ **Check after predicting**

Prediction: No values are visited and the final **output** is 0.

Reason: The **accumulator** starts at zero and an empty **loop** never changes it.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace four visits before printing the final total

> Initialization before the loop makes the empty case well-defined.

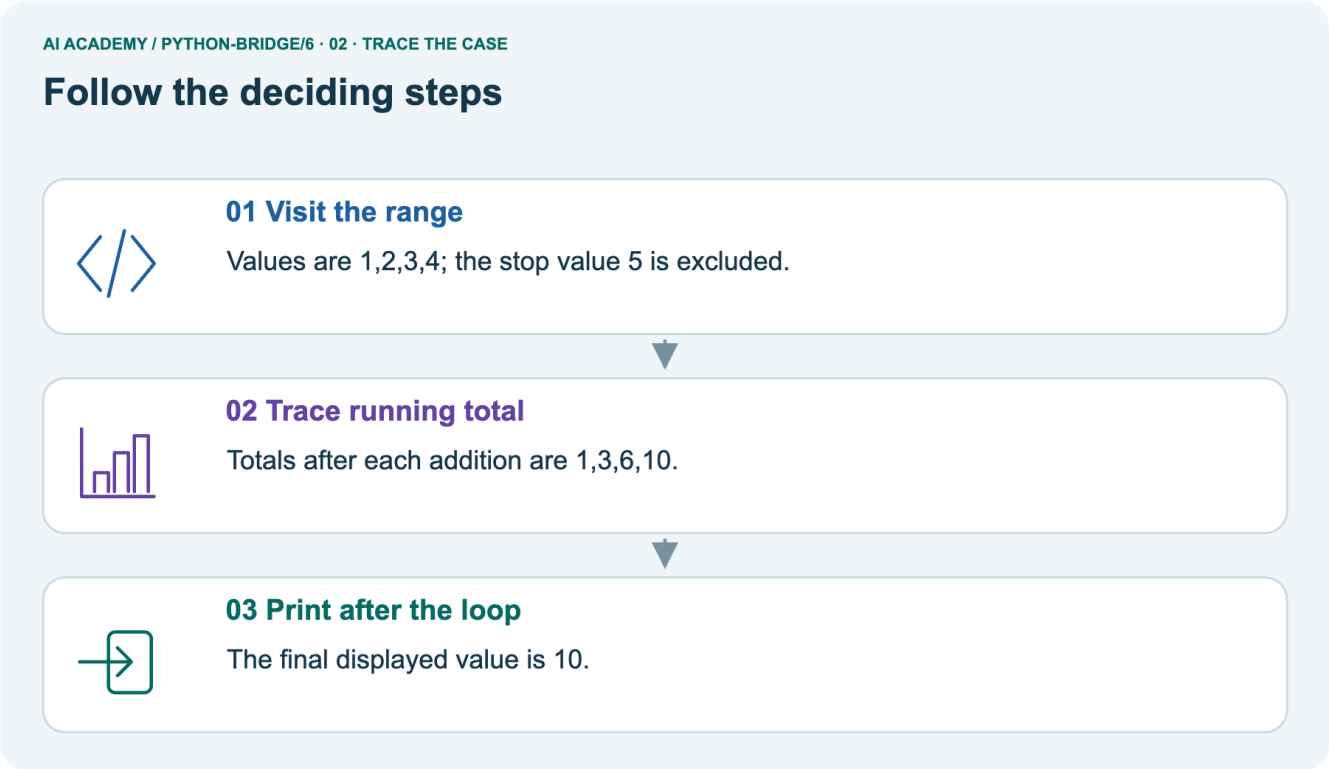
Read the pictured range and scope

Accumulate a short range. A running total adds the integers visited by range(1,5).

Python 3: total=0; for value in range(1,5): total+=value; print(total). The print is after the **loop**.

The warm-up range(1,4) visits 1,2,3 and totals 6. The pictured range(1,5) visits 1,2,3,4 and totals 10. W2 uses range(2,5), which visits 2,3,4 and totals 9. Keep these three records labelled instead of swapping final numbers. The selected independent range is different again. All supplied ranges use the default positive integer step; this lesson does not implement arbitrary range-input validation.

Trace the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

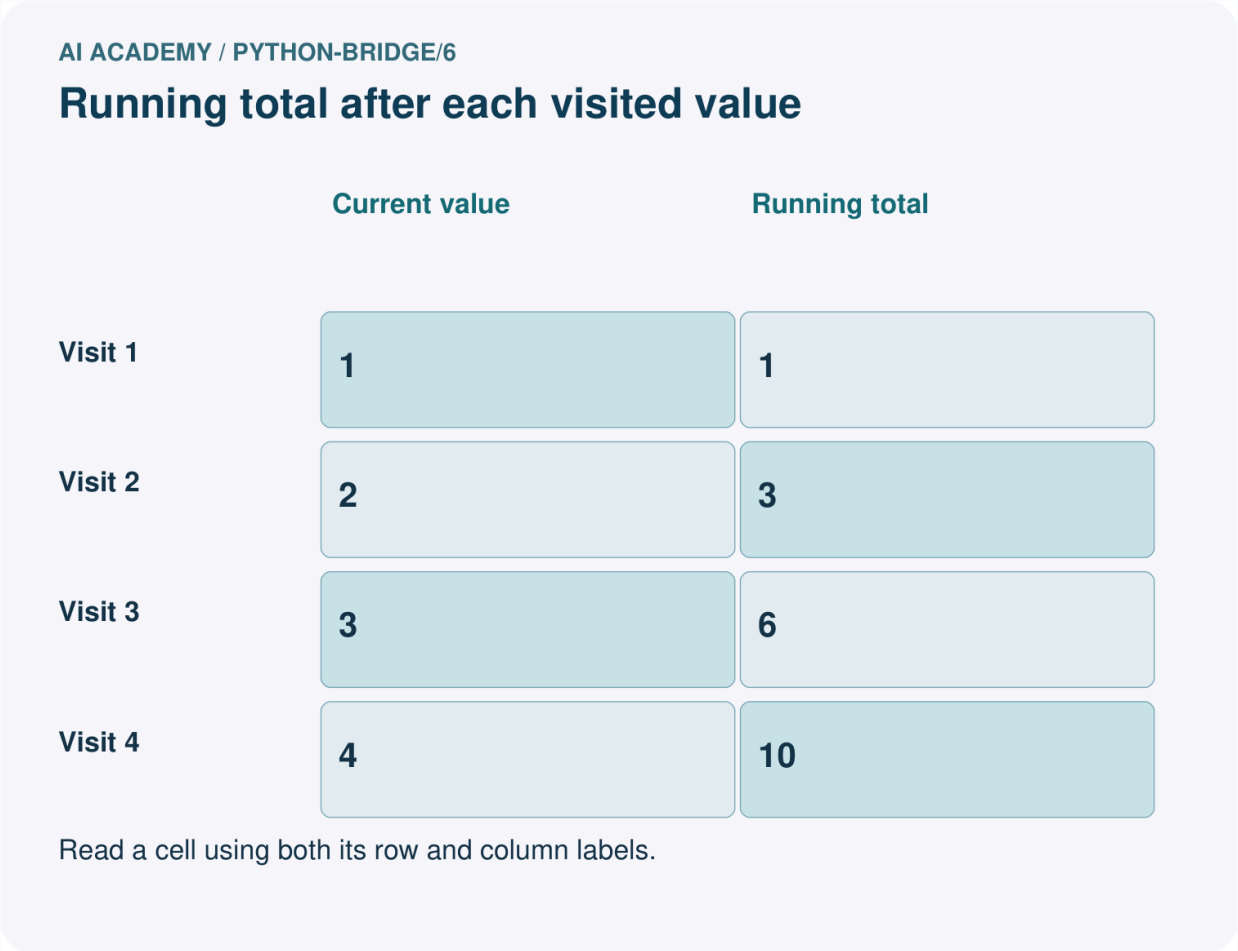
Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Visit the range	Values are 1,2,3,4; the stop value 5 is excluded.
2. Trace running total	Totals after each addition are 1,3,6,10.
3. Print after the loop	The final displayed value is 10.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

A **loop** repeats an operation for each visited value. A **trace table** distinguishes the current value from the **accumulator**; the range stop is not included.

Inspect the original current-value/running-total graphic



Follow the labelled picture with the matching explanation. Open the vector diagram

Match before/after state to complete code

Visited value	Total before	Total after	Output yet?
1	0	1	No
2	1	3	No
3	3	6	No

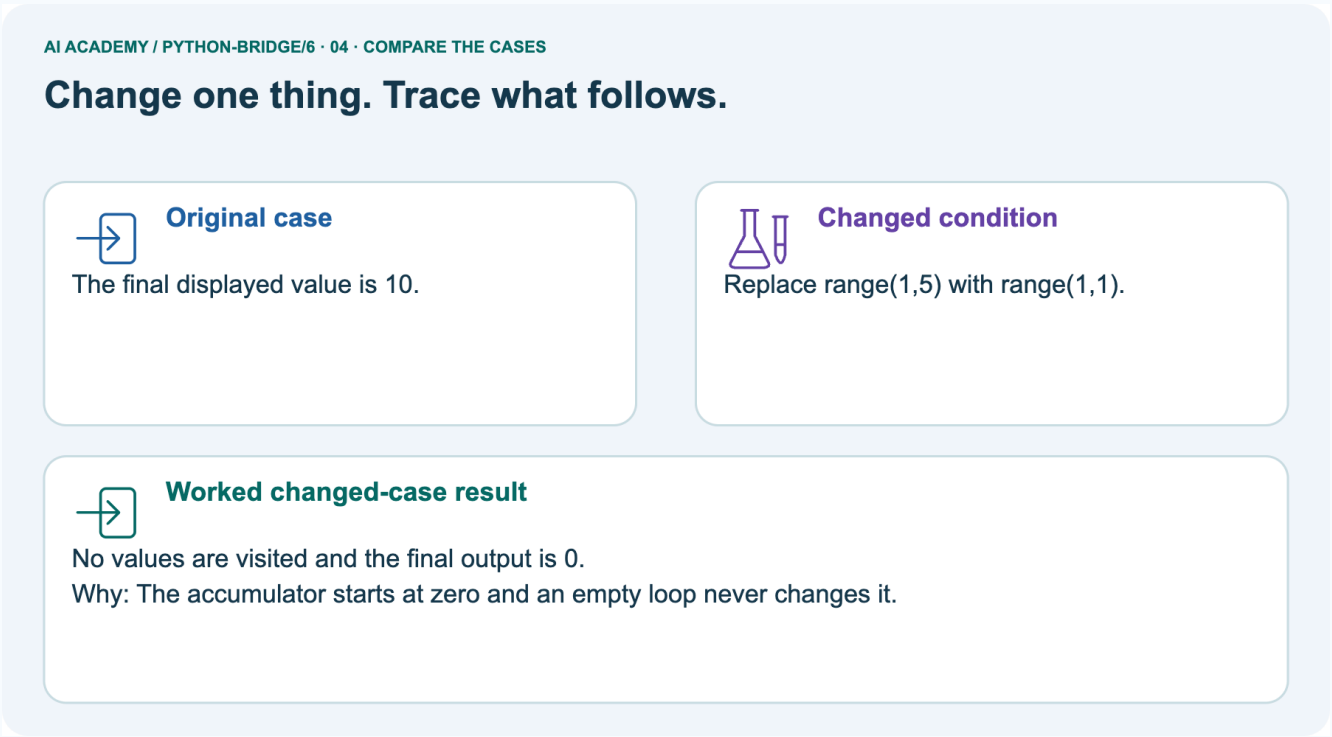
Visited value	Total before	Total after	Output yet?
4	6	10	No
After loop	10	10	Print 10 once

```
total = 0
for value in range(1, 5):
    total = total + value
print(total)
```

Predict replacing the range with an empty one

- Change one thing: Replace range(1,5) with range(1,1). Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

State written-rule and termination limits

- This bounded **loop** does not train an AI model; other **loop** conditions may require separate termination checks.
- Read the labelled picture: List the four visited values and their running totals. Explain why 5 is absent and what prints for range(1,1).

▼ Reference · Zero/one/many range forms

Range expression	Values supplied	Number of iterations
range(1, 4)	1, 2, 3	3

Range expression	Values supplied	Number of iterations
range(1, 2)	1	1
range(1, 1)	No values	0
range(4)	0, 1, 2, 3	4

With one argument, range starts at zero. range(4) supplies four values but none is 4. If the worked script uses range(1, 1), the body never runs and the initialized total remains 0.

Do not rely on a variable first assigned inside a **loop**: a zero-**iteration** run might never assign it.

> **Replace range(1, 4) with range(1, 3) in the worked script. Predict the supplied values and final total. What stop value is needed to include 3?**

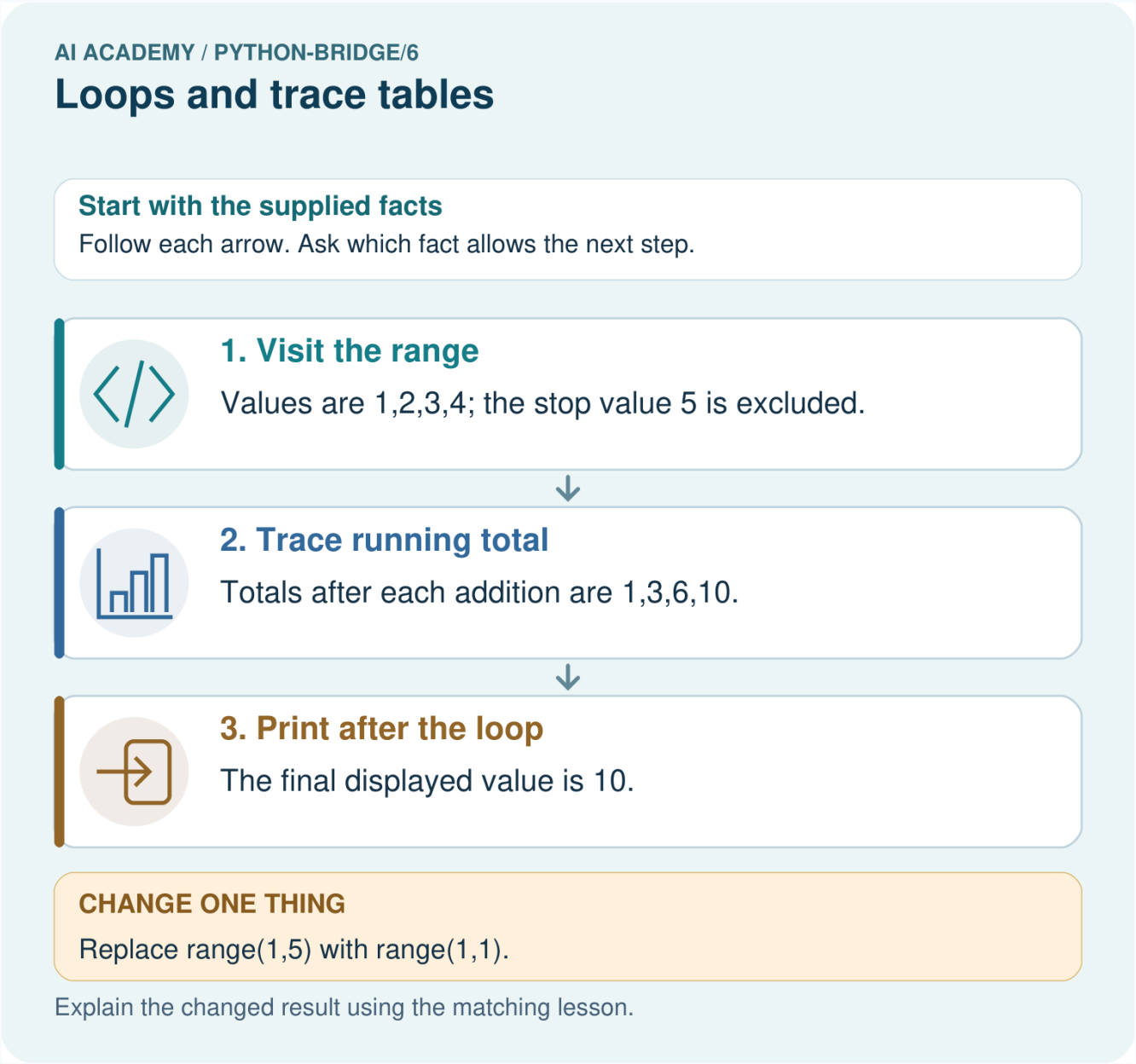
▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Loops and trace tables

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find the reset that discards earlier additions

> Moving a statement changes state or output events even when the final number sometimes agrees.

Read the complete faulty reset version

```
total = 0
for turn in range(1, 4):
    total = 0
    total = total + turn
print(total)
```

Trace lost state and the smallest repair

This faulty version prints 3. The extra reset discards earlier additions on every **iteration**. Remove the indented reset while keeping **initialization** before the **loop**.

Then test zero, one and several **iteration**s: the one-**iteration** case alone cannot reveal this reset bug.

Compare printing inside and after the loop

Moving print(total) into the correct **loop** produces three lines, 1 then 3 then 6. Moving it out produces only the final 6. Trace both values and **output** events.

A **loop** follows written instructions; repeating them does not make the program learn.

Explain the erroneous final total

- For the faulty script with range(1, 4), record total after each addition. Explain the wrong final **output** and remove only the faulty reset.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise the W2 range trace with less help

> Write each visited value before following the updates.

Explain the procedure

- List the values supplied by range(2, 5) and explain why 5 is absent. How many times does its body run?

Replace range(1, 4) with range(1, 3) in the worked script. Predict the supplied values and final total. What stop value is needed to include 3?

Explain your prediction to a partner before comparing results.

Complete the 2-through-4 W2 trace

Use workbook W2 for the connected example "**Loop**s and **trace table**s". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Check the excluded stop

- Use the worked case for **Loop**s and **trace table**s. Proposed explanation: "The stop value is included". Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the empty pictured range

- List the four visited values and their running totals. Explain why 5 is absent and what prints for range(1,1).

My explanation using a pictured step and the given facts

Reserve independent work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace before running

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check six complete loop-state and output cases

> List values, trace state and record each output event.

All supplied range and mutation cases

Complete supplied case	Visited values / expected output	Why test it
A · Correct script range(1,1)	No visits; final 0	Initialization must exist before a zero-iteration loop.
B · Correct script range(1,2)	Visit 1; final 1	One iteration alone hides the reset bug.
C · Correct script range(1,4)	Visit 1,2,3; final 6	Several iterations require accumulated state.
D · W2 range(2,5)	Visit 2,3,4; totals 2,5,9; final 9	Start changes and stop remains excluded.

Complete supplied case	Visited values / expected output	Why test it
E · Faulty reset version range(1,4)	After-addition totals 1,2,3; final 3	Earlier additions are erased.
F · Correct C with print moved inside	Output lines 1,3,6	Output-event location differs from final-only printing.

Complete W2 script

```
total = 0
for turn in range(2, 5):
    total = total + turn
print(total)
```

Predict, run or trace, then diagnose the first change

1. Predict the visited values and before/after totals for A–F before running or paper tracing. Use the complete W2 script for D, and keep it separate from the pictured 1-through-4 script. Test the faulty reset at zero, one and several iterations; explain why agreement on a tiny case does not vindicate the procedure.

   Move print only after preserving the original output record, and count the output lines as well as their values. If a variable is first assigned inside an empty loop, it may not exist in a fresh namespace; do not rely on leftovers from earlier runs. These are bounded deterministic loops, not learning algorithms. Keep the selected 2-through-5 transfer unused until the individual attempt.

▼ **Keep the original and faulty code as references**

```
total = 0
for turn in range(1, 4):
```

```
    total = total + turn
print(total)
```

```
total = 0
for turn in range(1, 4):
    total = 0
    total = total + turn
print(total)
```

Keep the evidence record

Copy the worked script. Change the range to range(2, 5). Record turn, total before and total after each **iteration**. Predict the single **output**, then run and compare.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected range and its empty variant

> The stop value is excluded; the accumulator survives an empty loop when initialized first.

Selected complete task and scope

List every value visited and each running total. Then predict the result for range(2,2).

```
total = 0
for value in range(2, 6):
    total = total + value
print(total)
```

Trace the exact selected range independently, recording visited values and running totals. Then use the stated empty-range variant from a fresh initial state. Do not import the warm-up or pictured stop value; record any prior exposure to the exact case before interpreting it as fresh transfer.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier loop transfer

A box starts with 5 counters. Each round adds 2 counters. Use rounds = 3, an initialized total and for turn in range(rounds). Print the final total once after the **loop**.

Trace the totals and predict fresh runs with rounds 0, 1 and 3. Assume rounds is a nonnegative integer.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Count iterations and a stopping point

Attempt this without hints

Use feedback to improve

Keep a before/after loop trace in the portfolio

> A final number alone can hide an incorrect path.

This week's contribution

Add a labelled **loop**s and **trace table**s **evidence** sheet to Python readiness portfolio. Show how you can trace variable values before and after each **loop iteration**.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

Finish: explain visited values, accumulated state and output position

> A correct final total needs a trace consistent with the loop body.

Explain without notes

- Close your notes. Explain this goal in your own words: Trace variable values before and after each **loop iteration**

> **My deciding step, evidence and one remaining question**

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Loop	A block of instructions that can run repeatedly
Iteration	One execution of a loop body
Accumulator	A variable that keeps a running result
Initialization	Giving a variable its starting value
Trace table	A record of variable values as instructions execute

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- List the values supplied by range(2, 5) and explain why 5 is absent. How many times does its body run?
- For the faulty script with range(1, 4), record total after each addition. Explain the wrong final **output** and remove only the faulty reset.
- In your counter script, range(-2) performs no **iteration**s. Does returning the initial 5 make negative rounds valid? Explain what the script checks and what it assumes.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- Write the expected range values and **trace table** before running. Keep predicted and observed **output** separate, then explain the first row where they differ.

  For a **loop** that should add a fixed amount repeatedly, zero **iteration**s should preserve the initial amount. One **iteration** checks one update; several check accumulation.

- In your counter script, range(-2) performs no **iteration**s. Does returning the initial 5 make negative rounds valid? Explain what the script checks and what it assumes.

> **A learner checks the reset bug using only range(1, 2) and sees the expected answer 1. Explain why this does not establish correctness, and give a revealing test with its expected correct output.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

State after each addition

TURN	TOTAL BEFORE	TOTAL AFTER
1	0	1
2	1	3
3	3	6

Initialize total to 0 before the loop. The unindented print runs once afterward.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Trace variable values before and after each loop iteration	Explain the deciding step, show a trace or test, and state one limit.
Use the excluded stop value in range correctly	Explain the deciding step, show a trace or test, and state one limit.
Explain initialization and test zero one and several iterations	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: Accumulate a short range. A running total adds the integers visited by range(1,5).

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Accumulate a short range. A running total adds the integers visited by range(1,5).

- Which labelled step matches this detail? The final displayed value is 10.
- 1. Trace running total
- 2. Print after the **loop**
- 3. Visit the range
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Print after the **loop** The final displayed value is 10. A **loop** repeats an operation for each visited value.

A **trace table** distinguishes the current value from the **accumulator**; the range stop is not included.

Changed-case question and hint

- Changed-case practice: Replace range(1,5) with range(1,1). Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. No values are visited and the final **output** is 0.

- 3. This one example guarantees correct results for every future **input**.

- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: No values are visited and the final **output** is 0. The **accumulator** starts at zero and an empty **loop** never changes it.

Repair and keep the record

Review Branching and boundary tests (python-bridge/5). Goal: Trace the selected branch in an if elif else chain. Short practice: Trace -2, 4 and 5 through the branches.

Explain why the negative check comes first and what label -2 gets if it is removed.. Return to this week and explain what you changed.

Review Expressions and operator order (python-bridge/3). Goal: Evaluate numeric expressions using Python's grouping rules. Short practice: Trace the operation order in both expressions.

Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

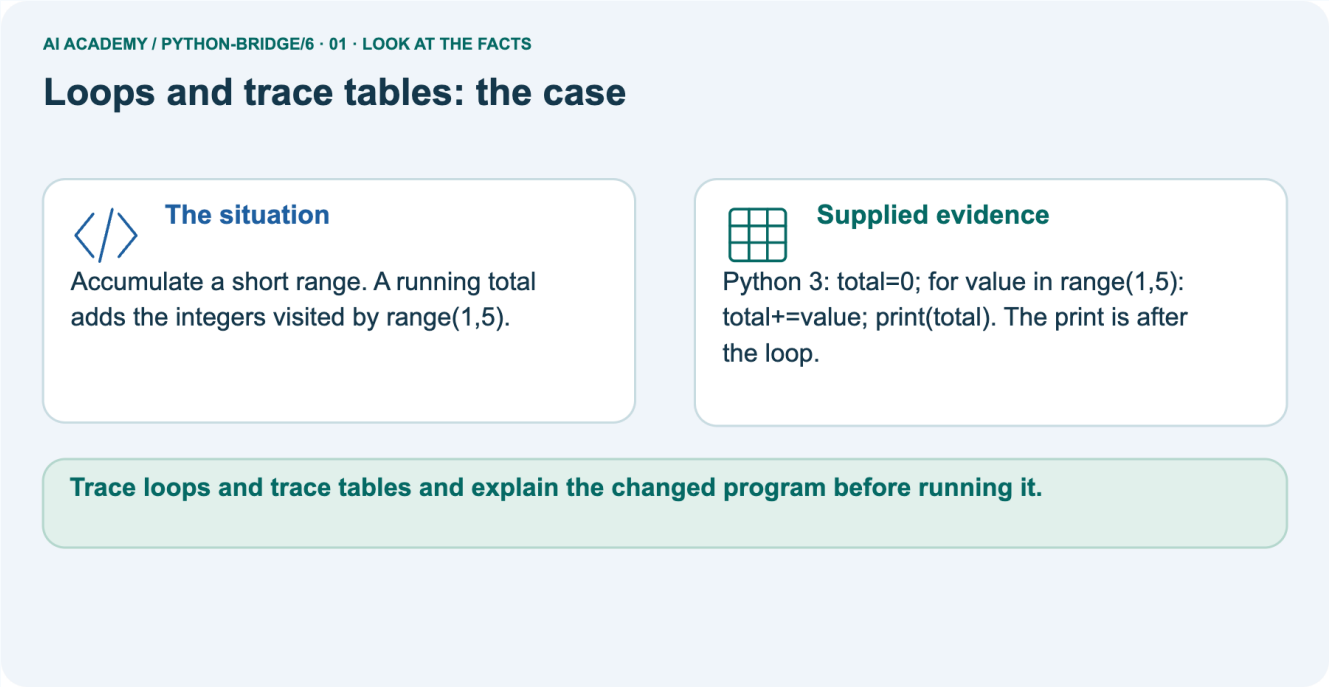
▼ Reference · The six-picture journey

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Loops and trace tables: the case

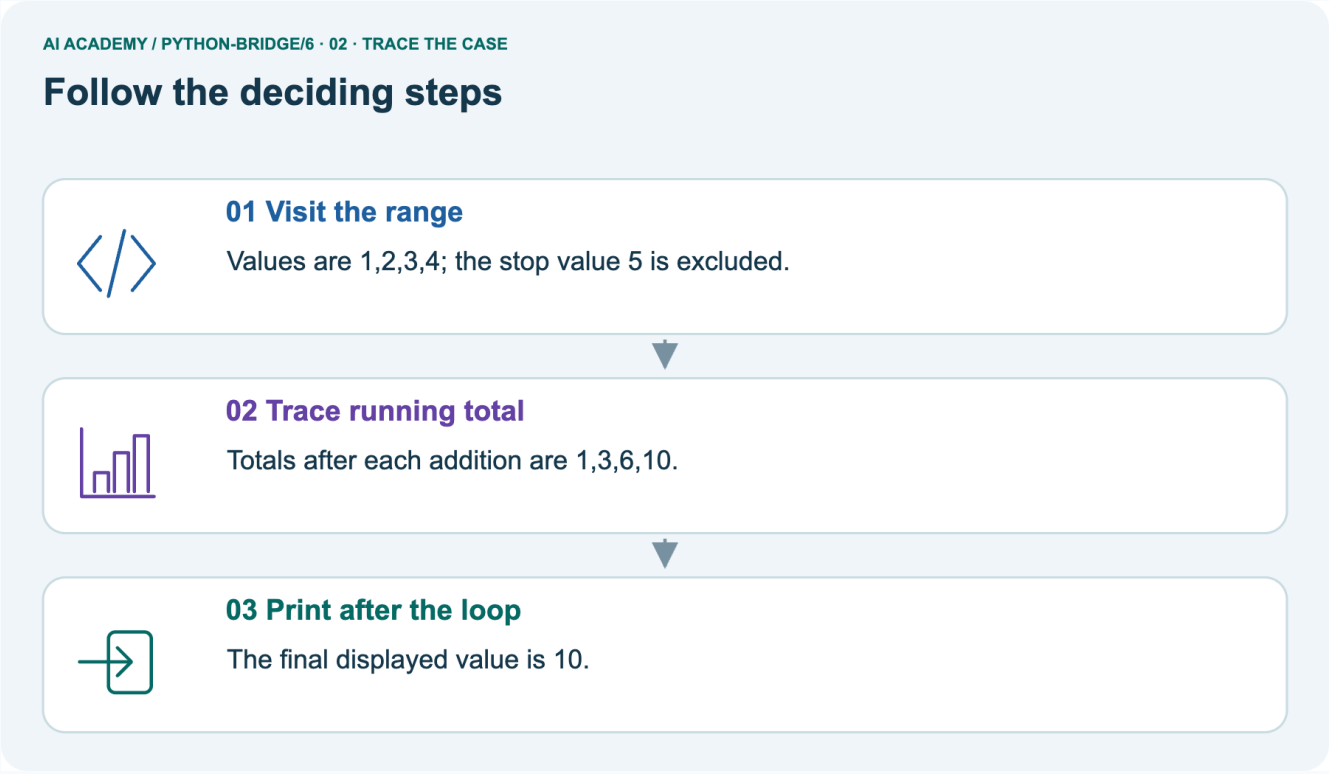


Follow the labelled picture with the matching explanation. Open the vector diagram

Accumulate a short range. A running total adds the integers visited by range(1,5). Supplied facts: Python 3: total=0; for value in range(1,5): total+=value; print(total). The print is after the **loop**.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Visit the range: Values are 1,2,3,4; the stop value 5 is excluded. → Trace running total: Totals after each addition are 1,3,6,10. → Print after the **loop**: The final displayed value is 10.

Point to each numbered stage and explain which recorded **input** it uses.

03. Running total after each visited value



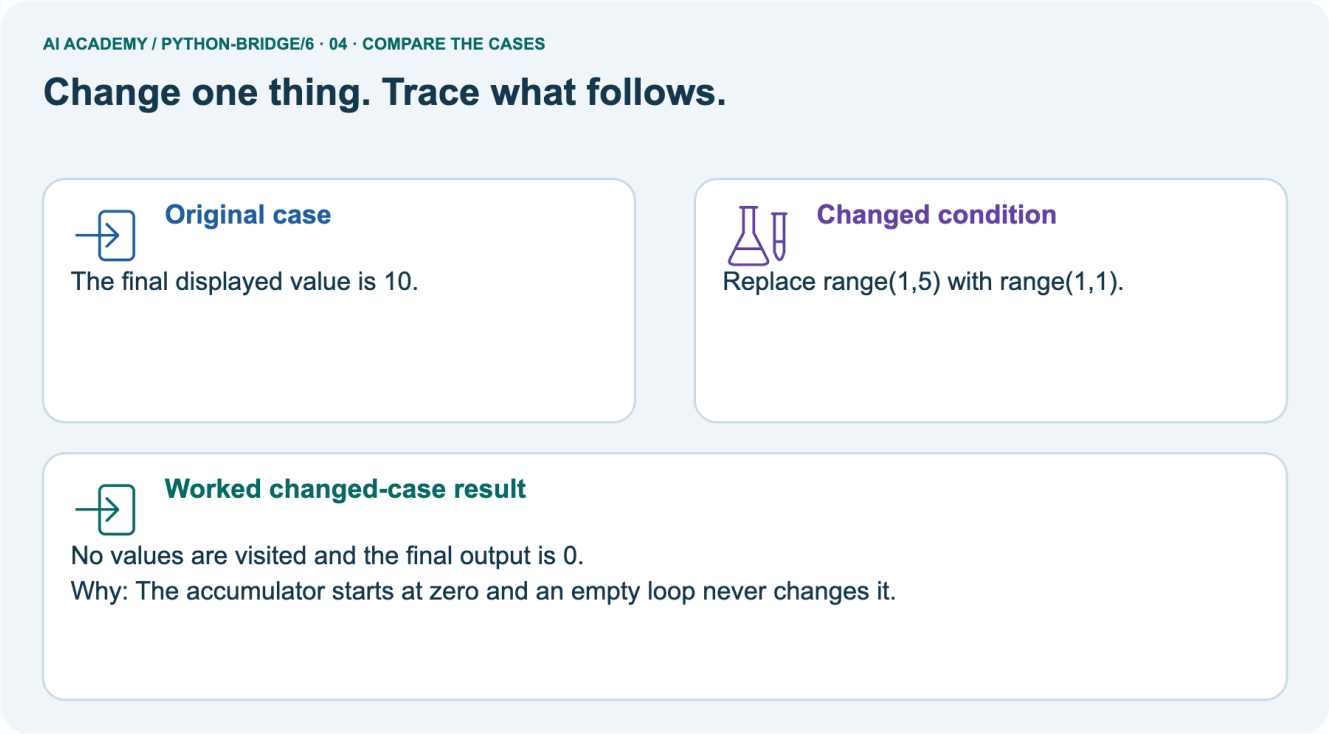
Follow the labelled picture with the matching explanation. Open the vector diagram

Running total after each visited value. Visit 1: Current value = 1, Running total = 1. Visit 2: Current value = 2, Running total = 3. Visit 3: Current value = 3, Running total = 6.

Visit 4: Current value = 4, Running total = 10.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



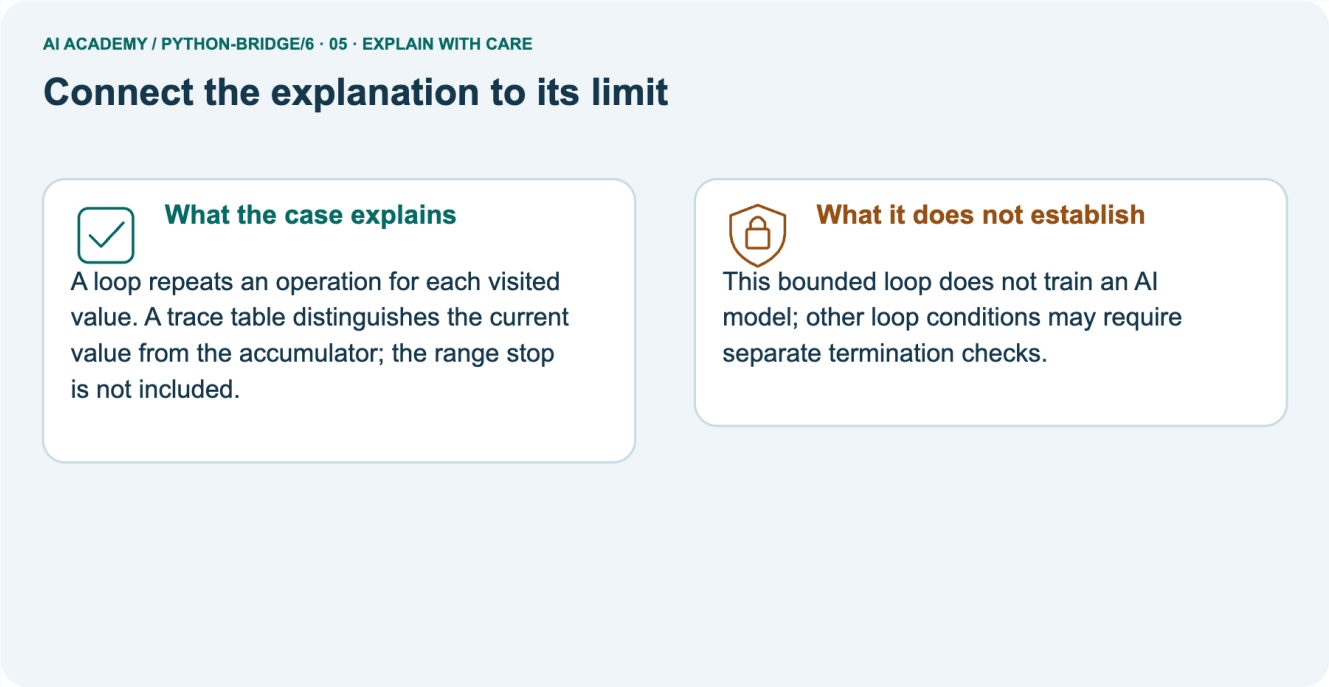
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: The final displayed value is 10. Changed condition: Replace range(1,5) with range(1,1). Expected result: No values are visited and the final **output** is 0.

Reason: The **accumulator** starts at zero and an empty **loop** never changes it.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



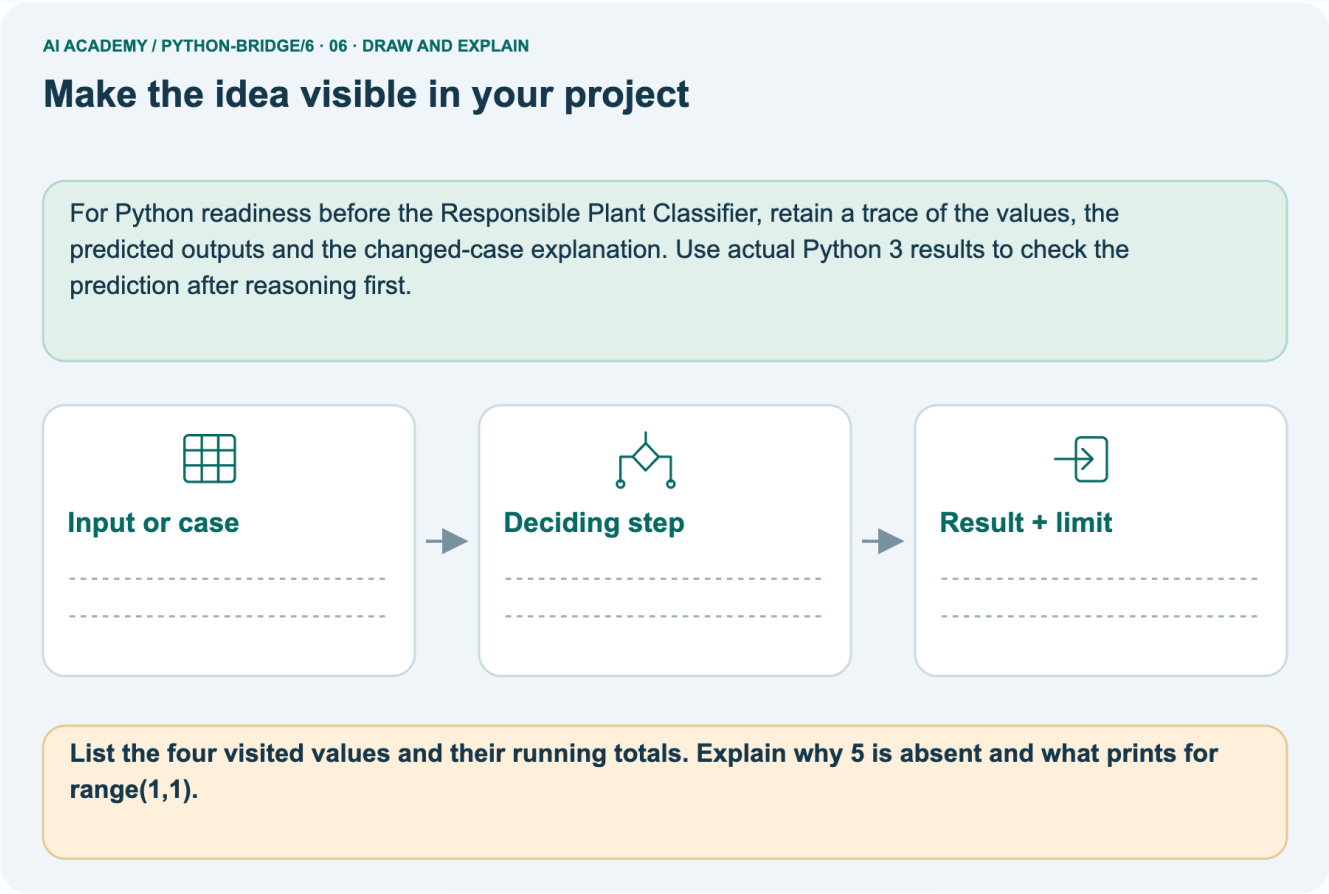
Follow the labelled picture with the matching explanation. Open the vector diagram

A **loop** repeats an operation for each visited value. A **trace table** distinguishes the current value from the **accumulator**; the range stop is not included.

Limit: This bounded **loop** does not train an AI model; other **loop** conditions may require separate termination checks.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: List the four visited values and their running totals. Explain why 5 is absent and what prints for range(1,1).

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Loops and trace tables: the case

02. Follow the deciding steps

03. Running total after each visited value

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.